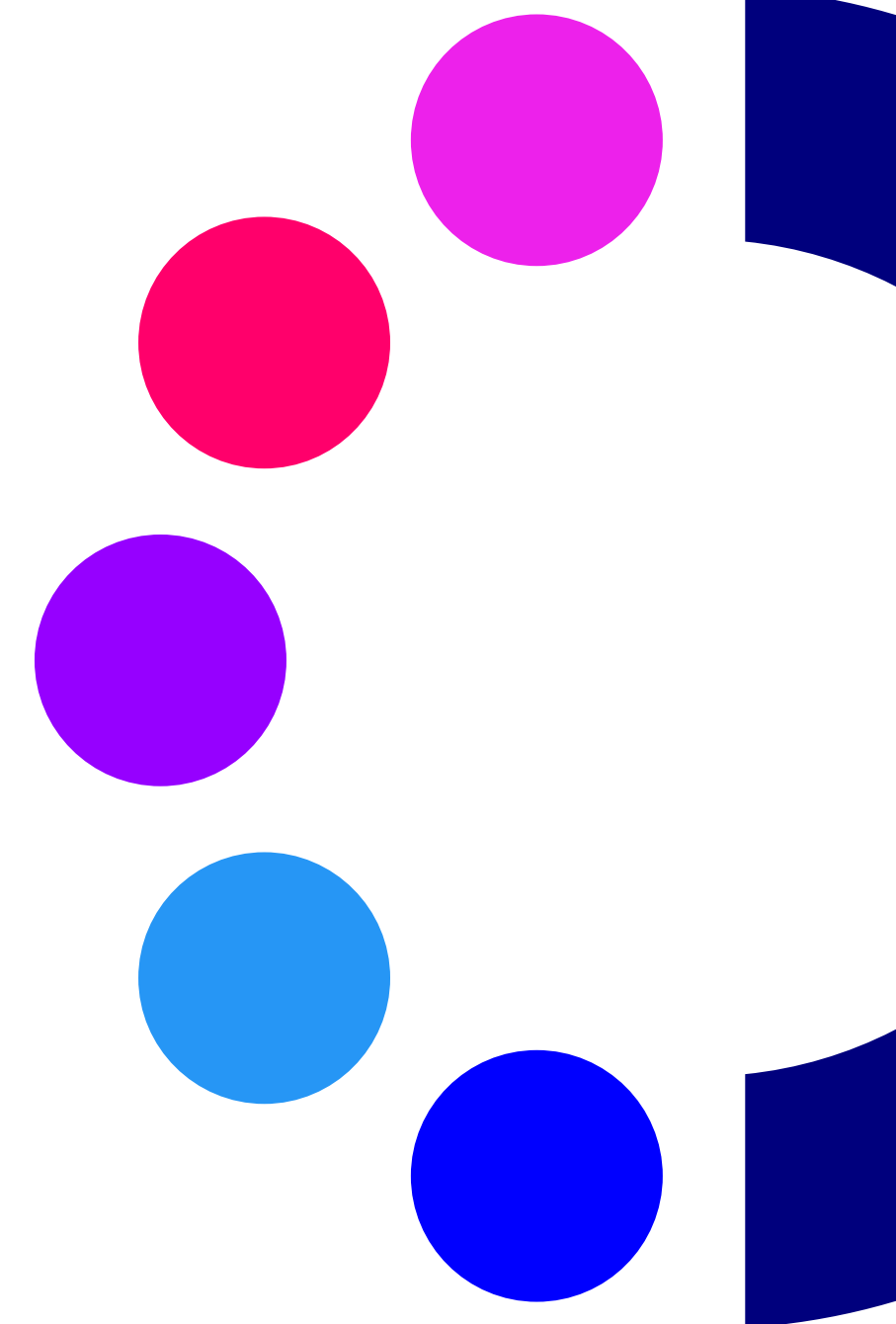


Real Time Performance Diagnosis in Distributed Databases

Partha Kanuparth, Alolita Sharma (AWS)
ApacheCon 2019



About

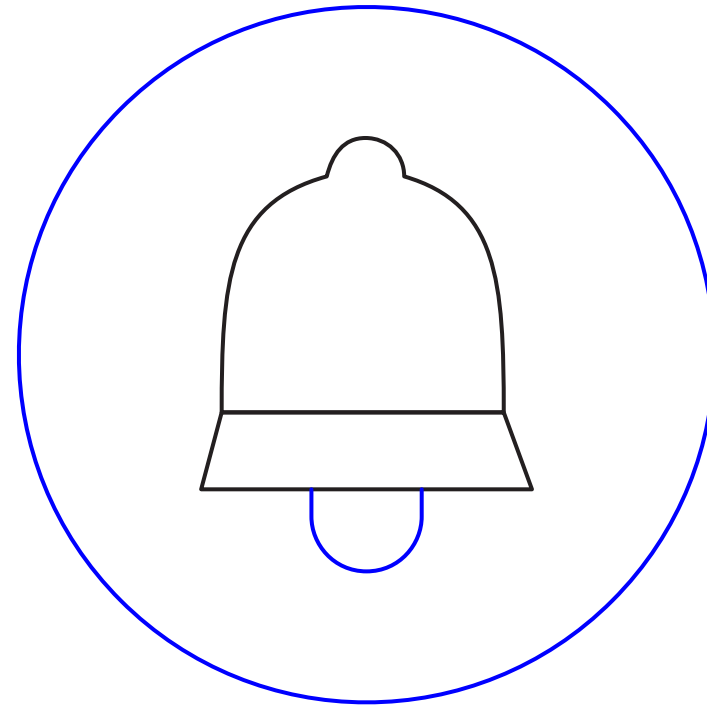
- Partha: Principal Engineer, AWS
- Alolita: Principal OS Technologist, AWS
- This is the work of a team: Adithya Chandra, Joydeep Sinha, Karthik Kumarguru, Palash Hedau, Prachi Kumar, Balaji Kannan, Bhaven Avalani, Kiran Dinakurthi, Partha Kanuparth.

Open Distro for Elasticsearch — Features



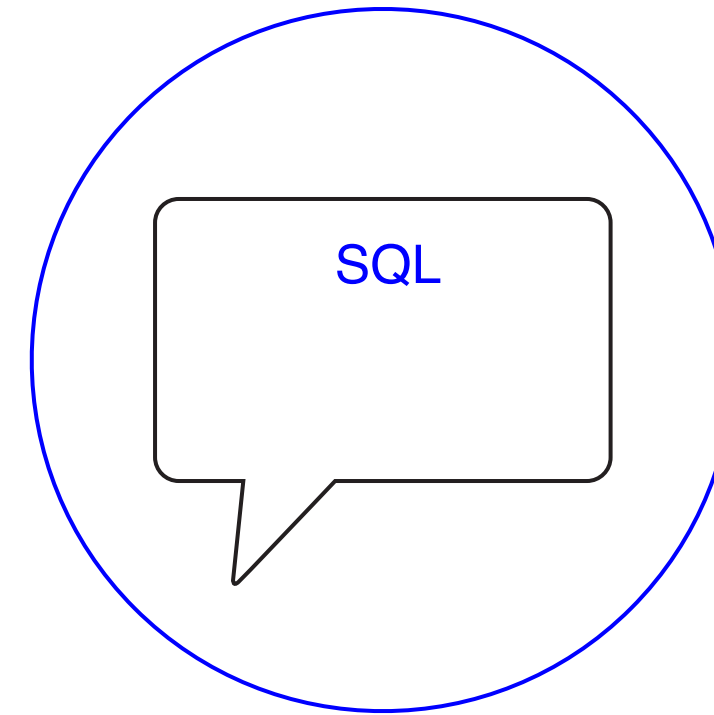
Security

Achieve encryption in-flight, role-based access control, audit logging, and compliance



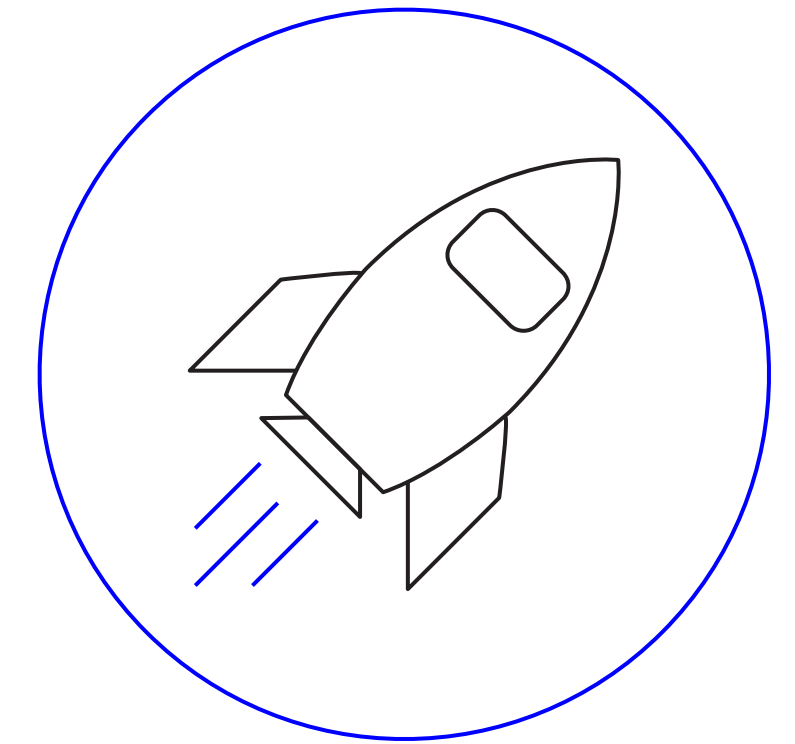
Alerting

Monitor your data and send automatic alerts on any changes in your data



SQL

Easily interact with your Elasticsearch cluster and extract insights using the familiar SQL query syntax



Performance Analyzer

Get deep visibility into system bottlenecks even when your Elasticsearch cluster is under duress

Get Involved in the Project

PerfTop is a fully open source project and its success is driven by the community's participation, contributions and innovation to the project.

You can follow project discussions, help fellow community members on the project forums, write a technical blog post on your use of PerfTop, contribute PRs, file bugs or request a feature at:

Discussion Forums: **discuss.opendistrocommunity.dev**

Code Community: **github.com/opendistro-for-elasticsearch/community/issues**

Project Website: **opendistro.github.io/for-elasticsearch/features/analyzer.html**

PerfTop: **github.com/opendistro-for-elasticsearch/perftop**

Performance Analyzer: **github.com/opendistro-for-elasticsearch/performance-analyzer**

This talk

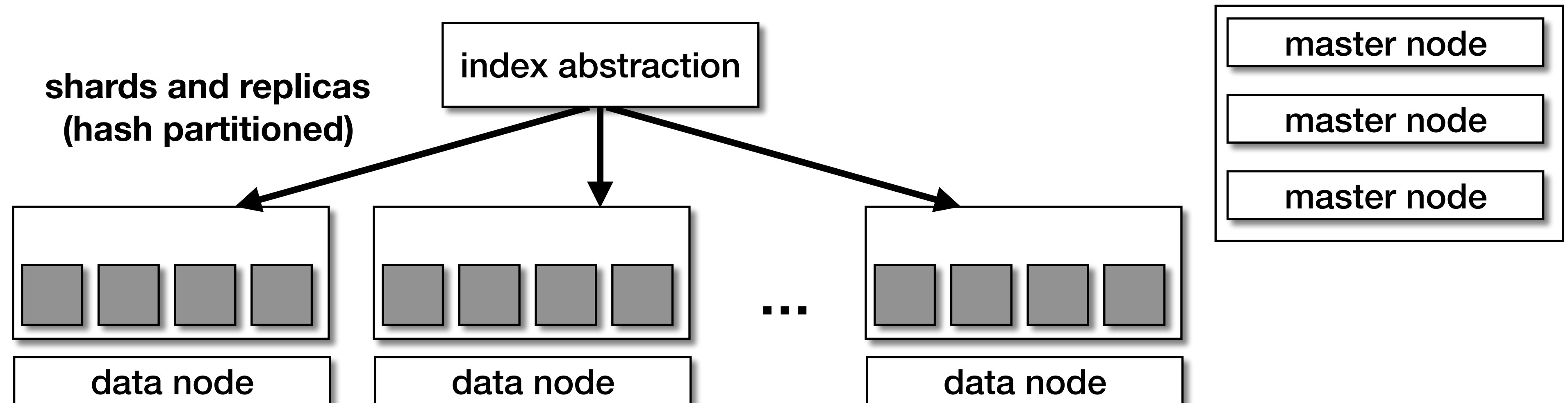
- Observability
- Root cause analysis

Elasticsearch

- eventually consistent data store
- no ACID semantics
- focus on real-time guarantees (data freshness)
- mainly log analytics: write-heavy, append-only; canned queries

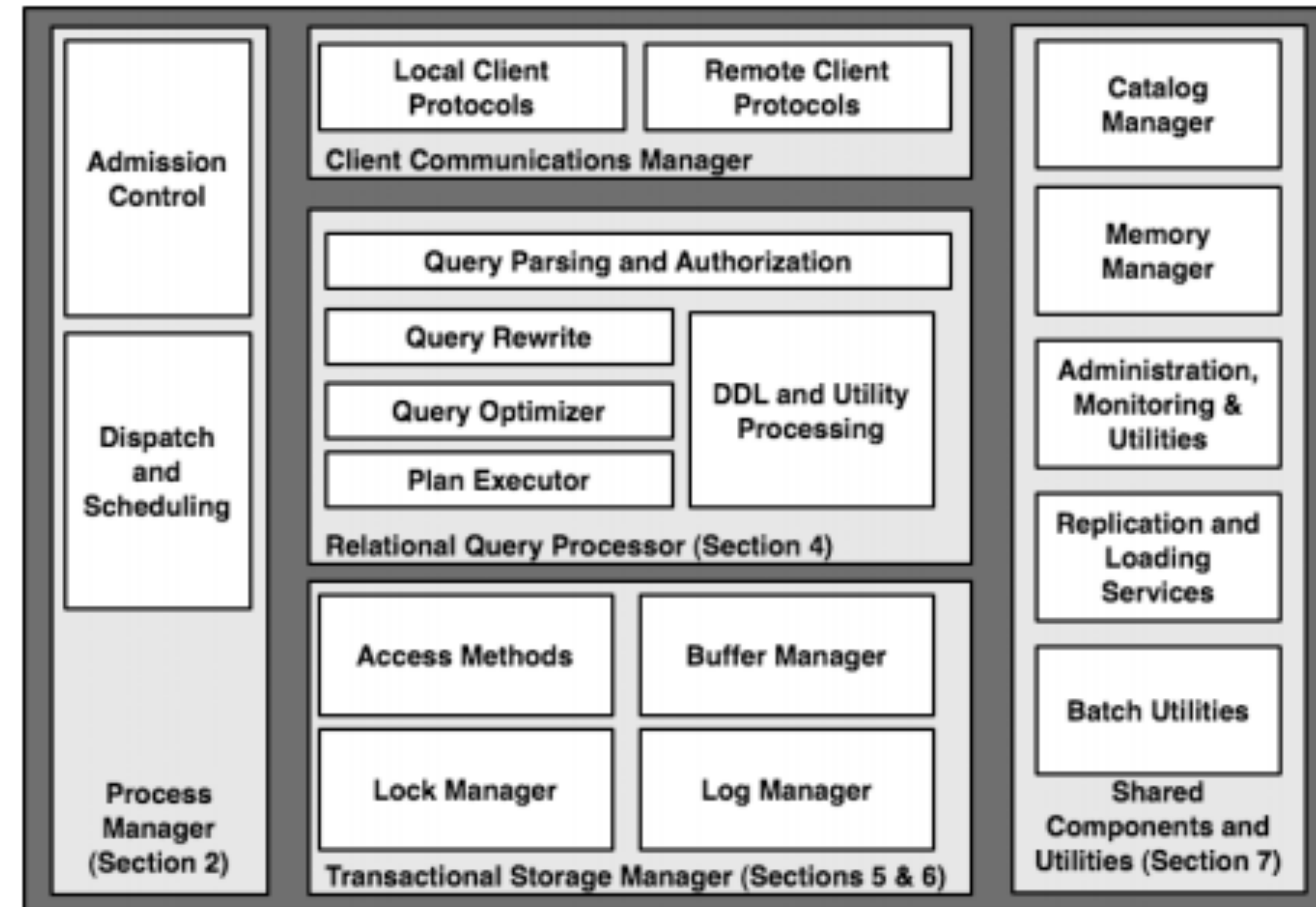
APIs:

- ingest: doc and bulk
- query
- data and cluster management



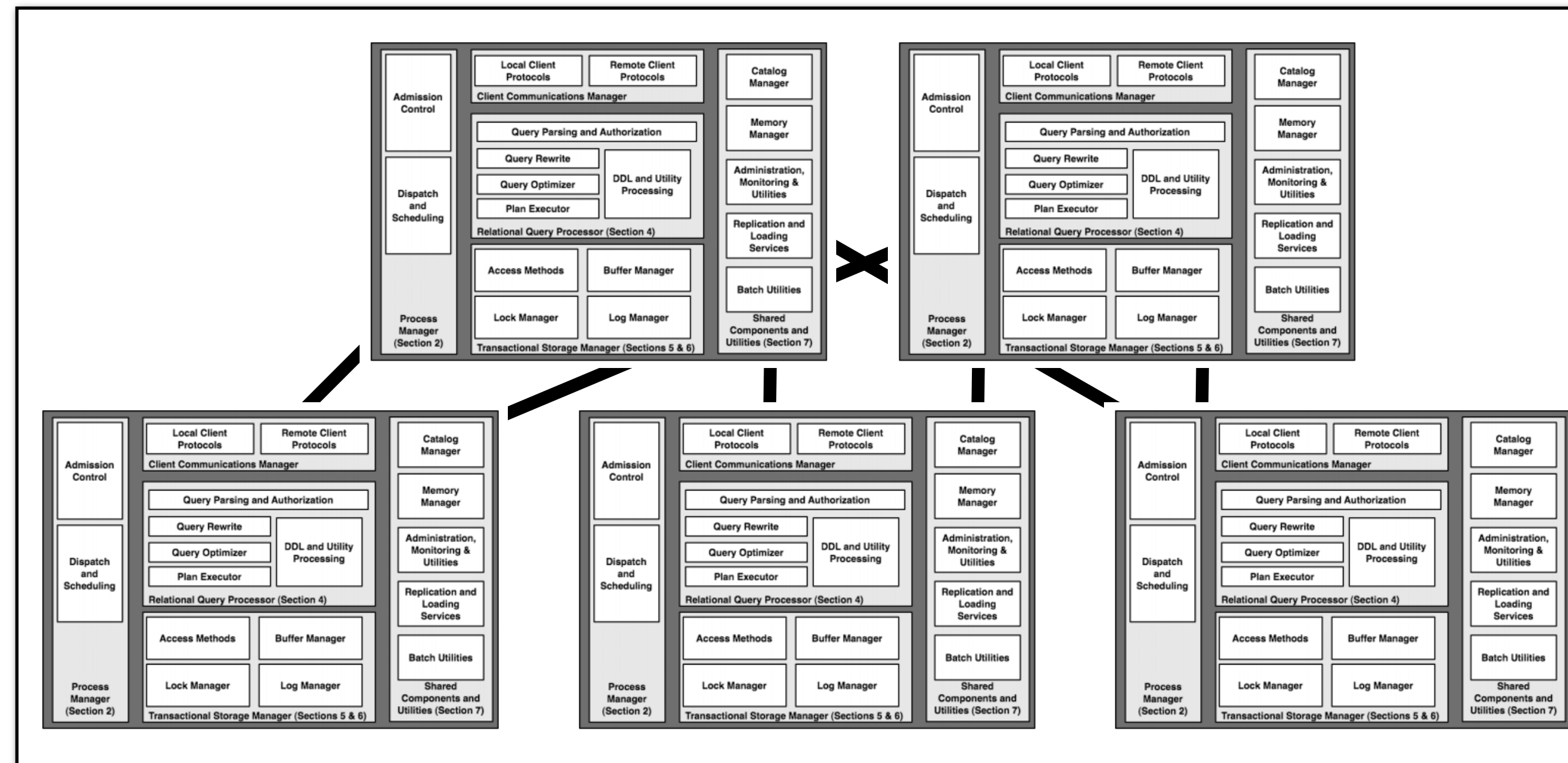
Database diagnosability

- Complex systems!
- Not built for performance *diagnosability*
 - Is there a problem?
 - Why is my query slow?
 - Why is my ingestion slower?
 - How to (re-)provision cluster?



Source: Hellerstein et al. 2017

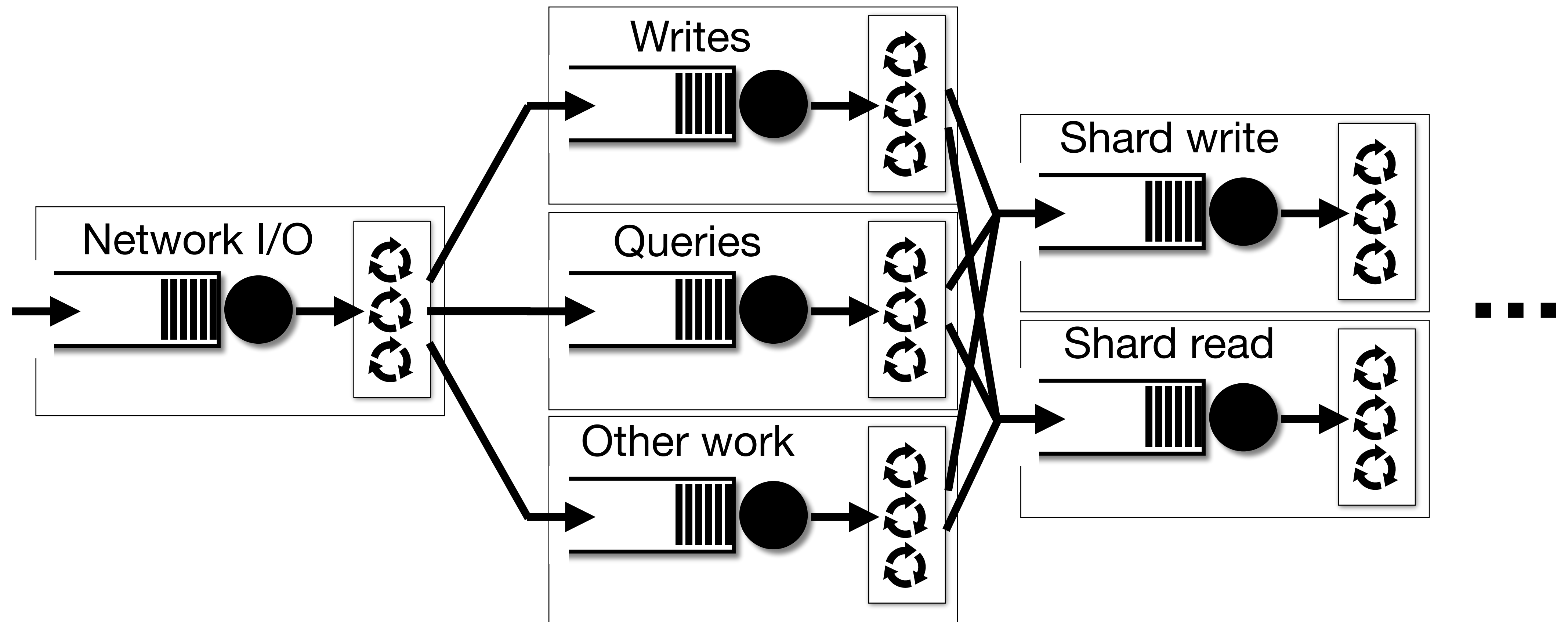
Distributed Database diagnosability



- Combinatorial performance problem modes
- Problems span multiple layers

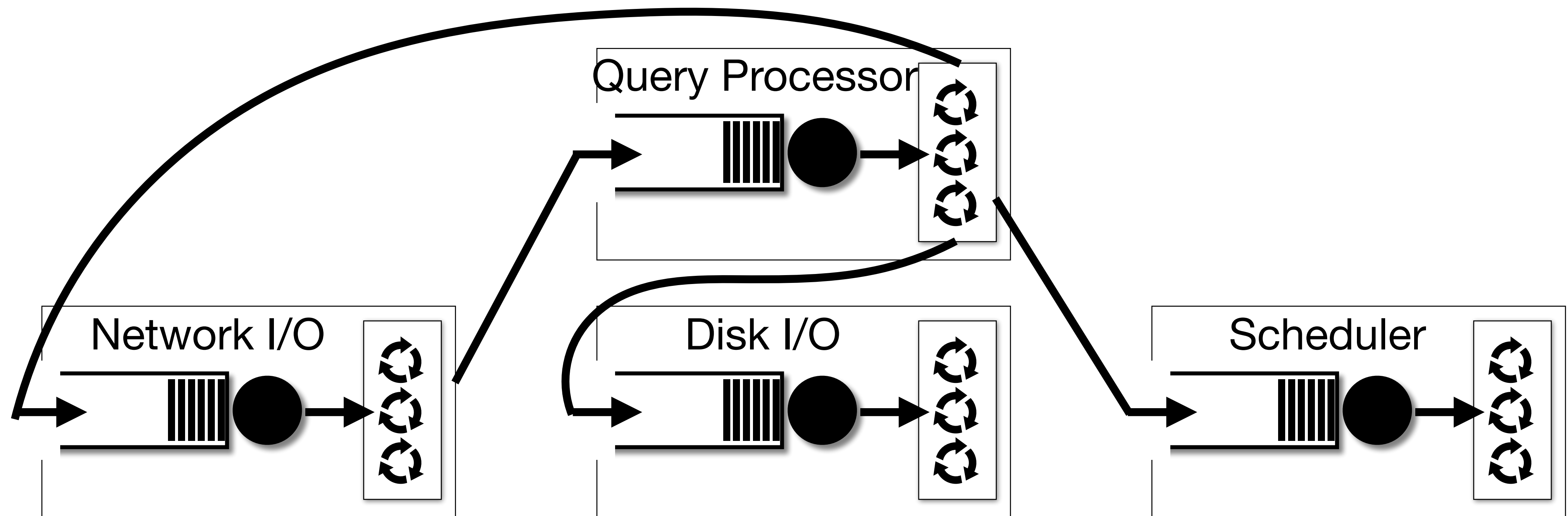
Low-level architectures

- Staged and functional (e.g., Elasticsearch): enables scalability



Low-level architectures

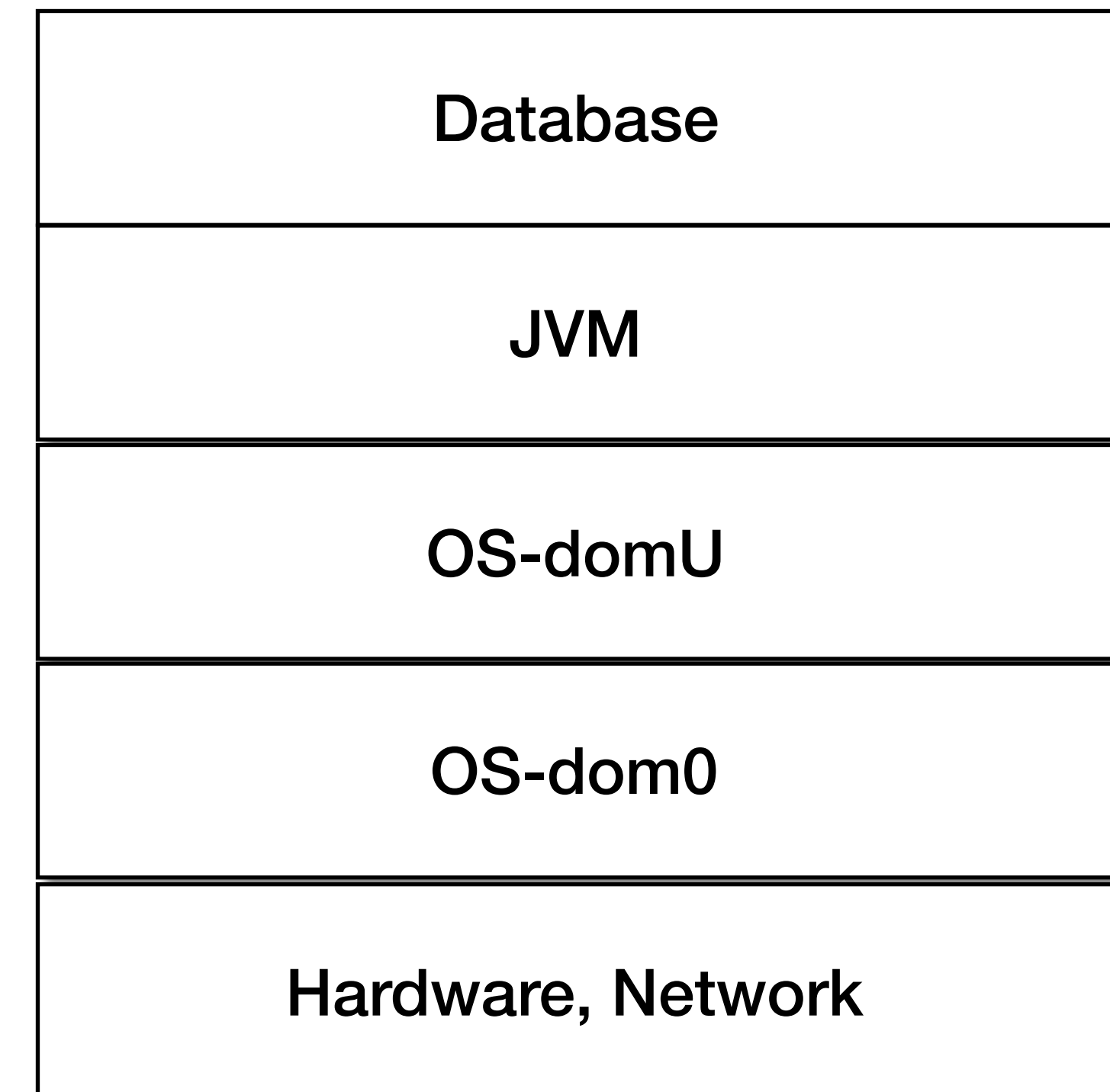
- Functional and resource-based: enables performance isolation (e.g., Redshift, Postgres) https://docs.aws.amazon.com/redshift/latest/dg/c_internal_arch_system_operation.html



both architectures: each thread executes atomic units of work from queue

Real Time Performance Diagnosis

- Goal: A system to:
 - accurately pinpoint current performance problem(s)
 - measure fine-grained resource usage
 - across the stack
 - across all resources: **physical**, **virtual**, **abstract**



hardware, network,
devices, queues

hypervisor,
process, thread,
JVM heap, v-devices

DB shards, indexes,
replicas

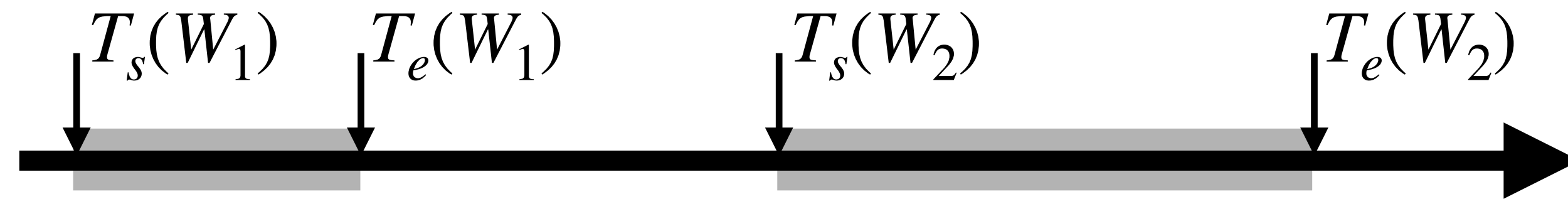
Diagnosis: Typical approaches

- Databases typically expose *informational* metrics
- We design for metrics for *diagnosability*
 - focus on resource usage (all types)
 - fine-grained breakdown: workload/operation type, time, data, ...

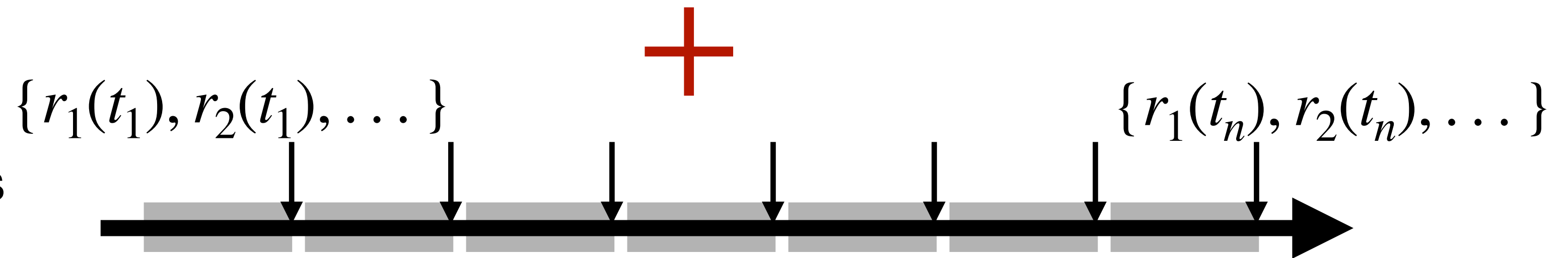
Observability

- Goal: Measure fine-grained resource usage of DB work
- High-level idea: construct event and utilization timelines; join

Thread i :
Events for units of work
 $\{W_1, W_2\}$: e.g., queries



Thread i :
resource measurements
(e.g., CPU util.)



Unit of work W_i :
resource measurements

$$W_i \rightarrow \{r_1(t) \dots\}, \{r_2(t) \dots\}, \dots$$

DB interfaces

- Lightweight library to capture events on each thread

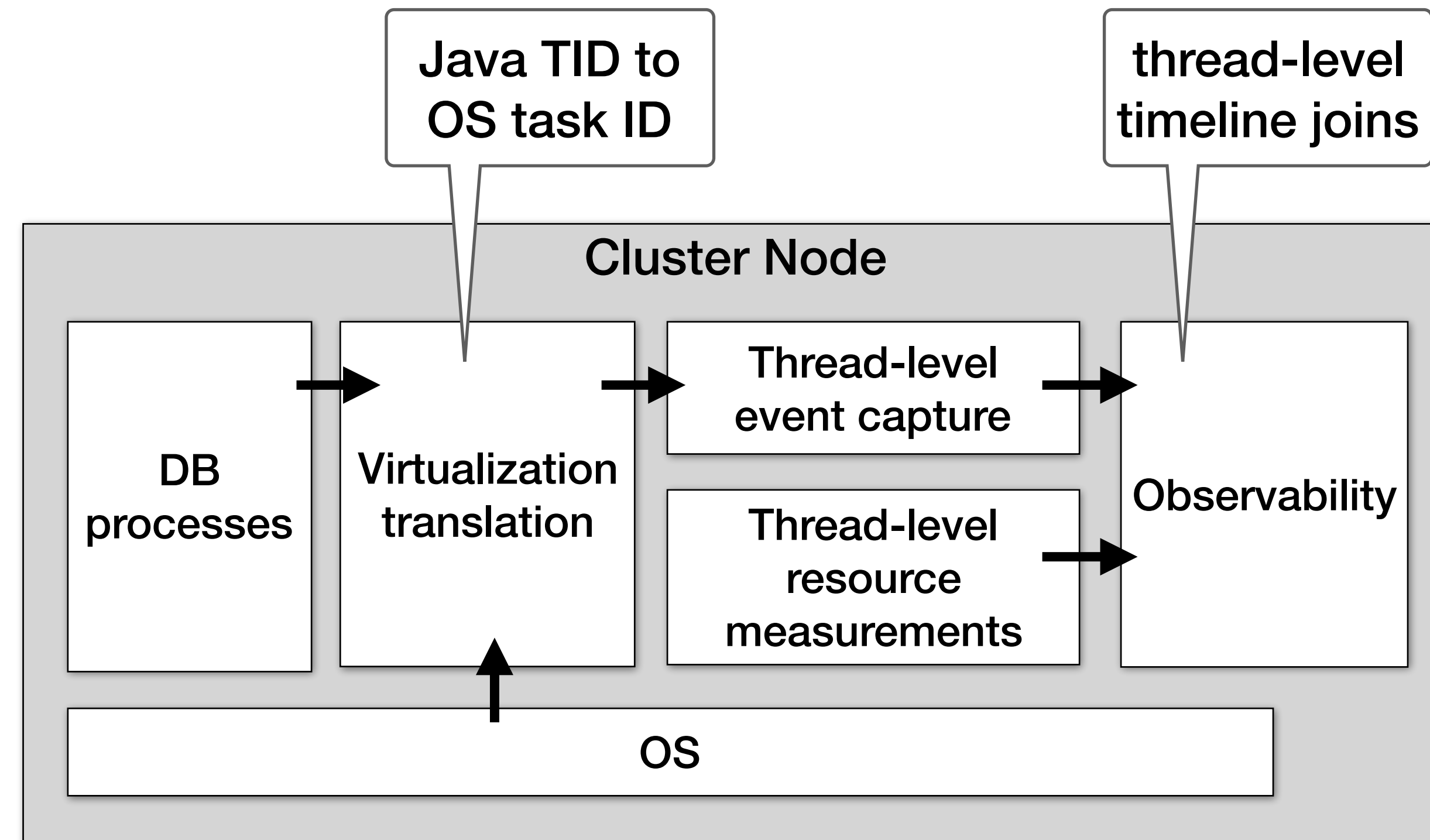
`AddEventForThread( event_type, event_metadata ... )`

e.g., query Q start

e.g., index I , shard S

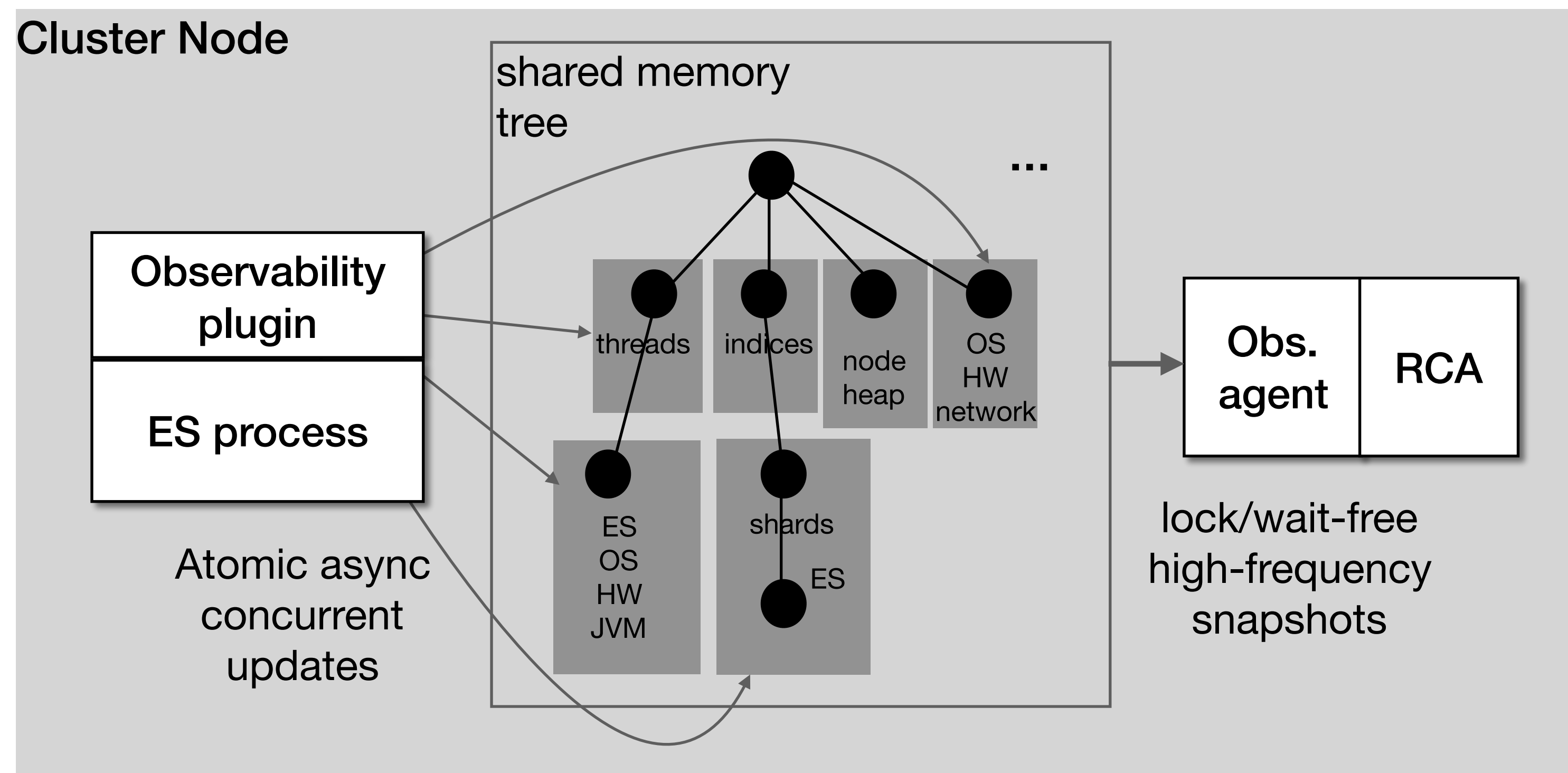
- Captures per-thread stack traces (via JVM) for sampling profiles
 - helps when not all events/threads are monitored (for events)

Logical organization



Implementation

- Agent: Process isolation from Elasticsearch
- v2: replaced tree with append-only log for throughput



Metrics API

- Output of Observability: returns last 5s of system view

`GetMetric ( metric, dimensions... )`

e.g., CPU utilization

e.g., index name, shard ID, node ID

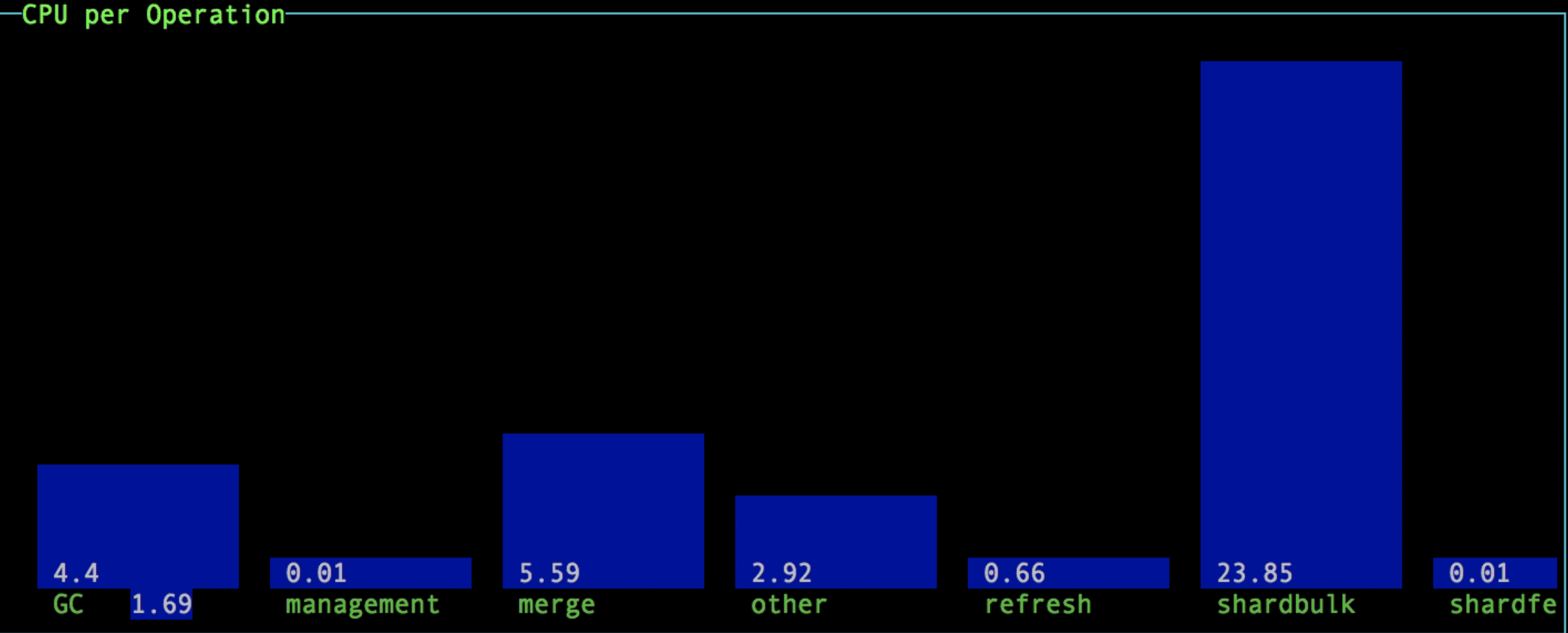
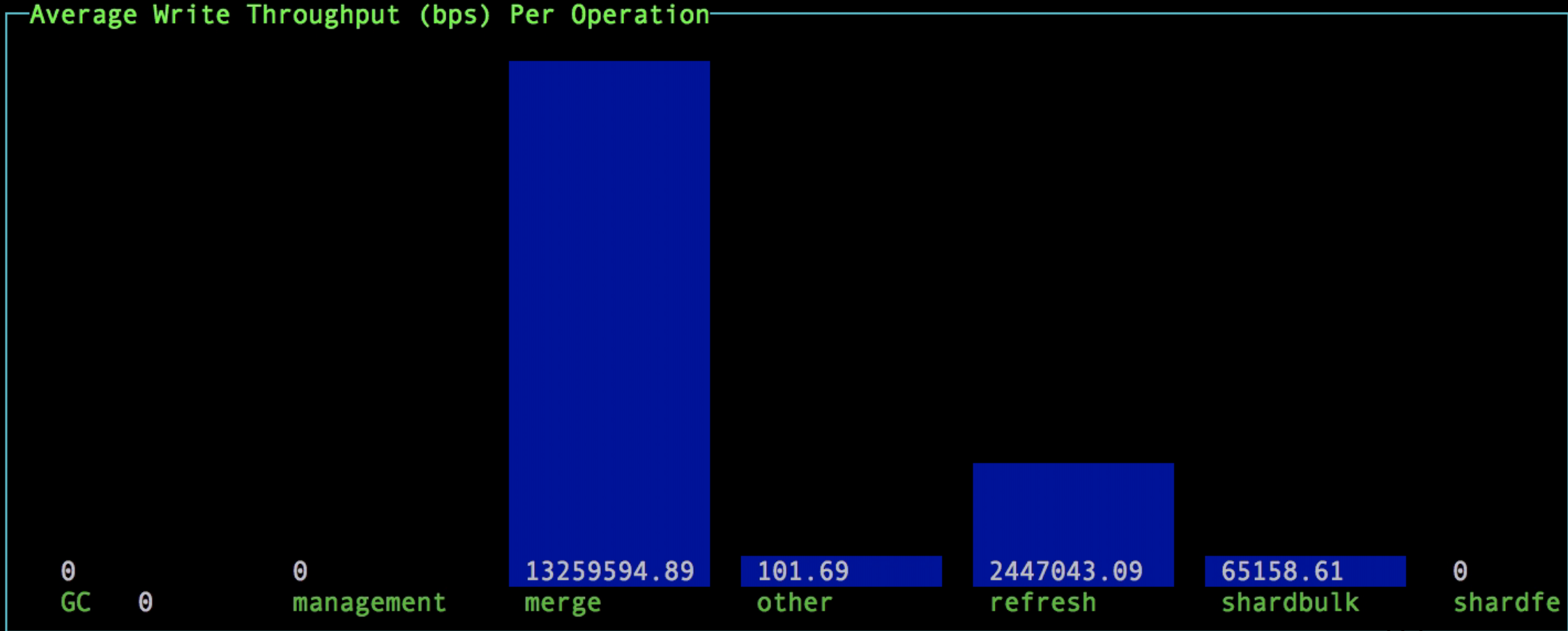
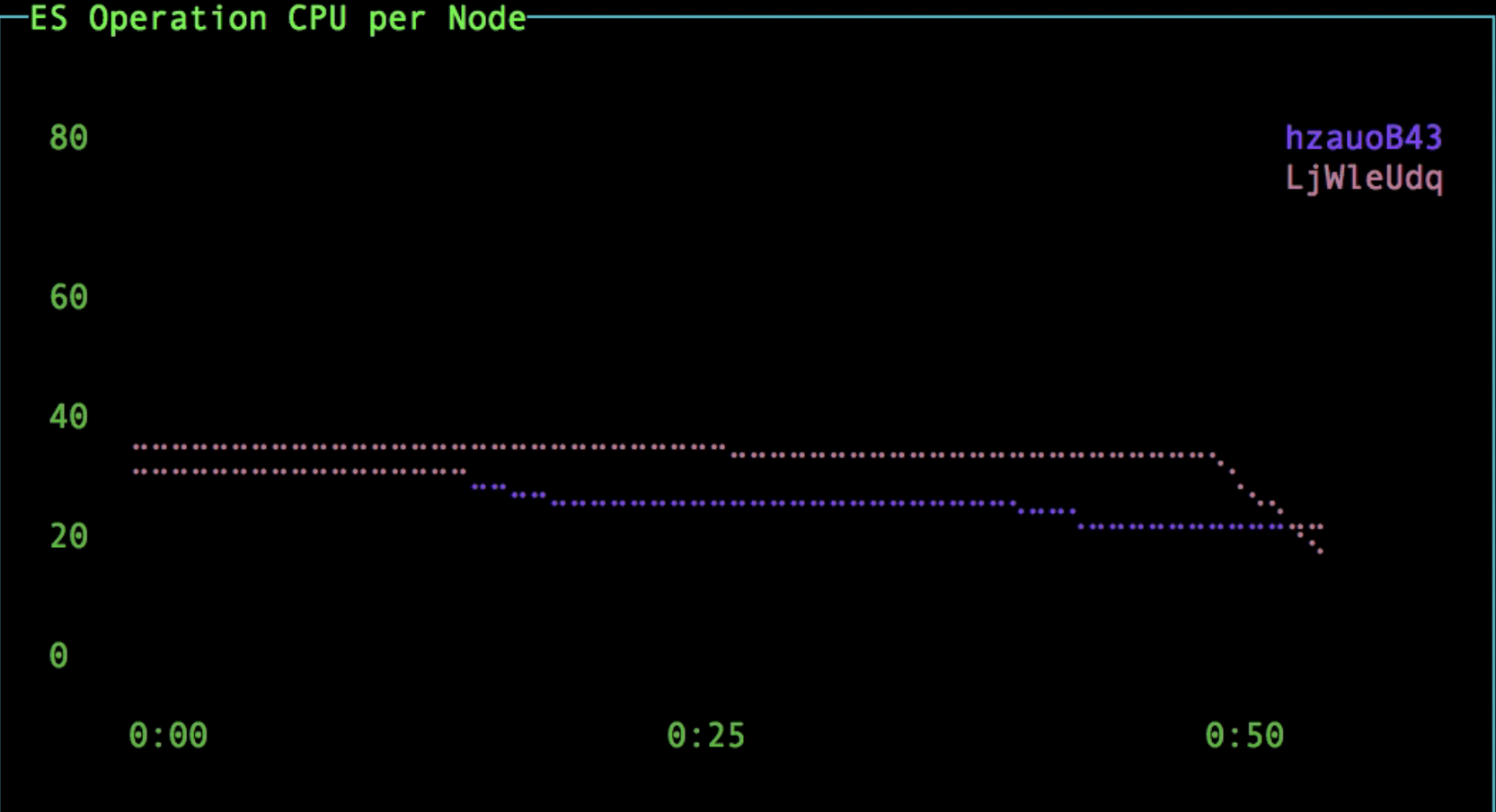
CPU utilization	Shard ID	Index name	Node ID

“get paging activity and disk I/O for bulk activity on a node”
“get CPU and heap allocations for search query”
“get GC CPU and page faults for a node”

PerfTop

Resource Metrics						
operation	index	shard	cpu	avgReadThroughputBps	avgWriteThroughputBps	node
merge	nyc_taxis	10	0.81	0	19162542.8	LjWleUdqSZKExh0k7ia
merge	nyc_taxis	6	0.76	0	13795881.71	LjWleUdqSZKExh0k7ia
shardbulk	nyc_taxis	24	0.76	0	17436410.87	LjWleUdqSZKExh0k7ia
merge	nyc_taxis	4	0.73	0	14057907.06	LjWleUdqSZKExh0k7ia
shardbulk	nyc_taxis	26	0.71	0	17128893.59	LjWleUdqSZKExh0k7ia
merge	nyc_taxis	2	0.71	0	11450013.28	LjWleUdqSZKExh0k7ia
merge	nyc_taxis	8	0.68	0	13728315.43	LjWleUdqSZKExh0k7ia
shardbulk	nyc_taxis	28	0.65	0	14366378.8	LjWleUdqSZKExh0k7ia
merge	nyc_taxis	16	0.65	0	10982184.01	LjWleUdqSZKExh0k7ia
merge	nyc_taxis	18	0.63	0	12109447.97	LjWleUdqSZKExh0k7ia
merge	nyc_taxis	14	0.63	0	10790466.85	LjWleUdqSZKExh0k7ia
shardbulk	nyc_taxis	61	0.54	0	3771860.57	hzauoB43SLCo1LUDsF6
shardbulk	nyc_taxis	15	0.52	0	3660108.69	hzauoB43SLCo1LUDsF6
shardbulk	nyc_taxis	11	0.51	0	3576194.27	hzauoB43SLCo1LUDsF6
shardbulk	nyc_taxis	5	0.5	0	3551458.45	hzauoB43SLCo1LUDsF6
shardbulk	nyc_taxis	41	0.5	0	3520545.15	hzauoB43SLCo1LUDsF6
shardbulk	nyc_taxis	35	0.5	0	3529262.39	hzauoB43SLCo1LUDsF6

Workload					
operation	status	indices	latency	itemCount	node
bulk	200	0	302.77	897711	hzauoB43SLC
bulk	200	0	274.93	381878	LjWleUdqSZK
search	200	admin	171	0	hzauoB43SLC
search	200	nyc_taxis	245	0	hzauoB43SLC
search	200	nyc_taxis1	450	0	hzauoB43SLC
search	200	sonested	1465.5	0	hzauoB43SLC
search	200	sonested	1876	0	LjWleUdqSZK



Root Cause Analysis

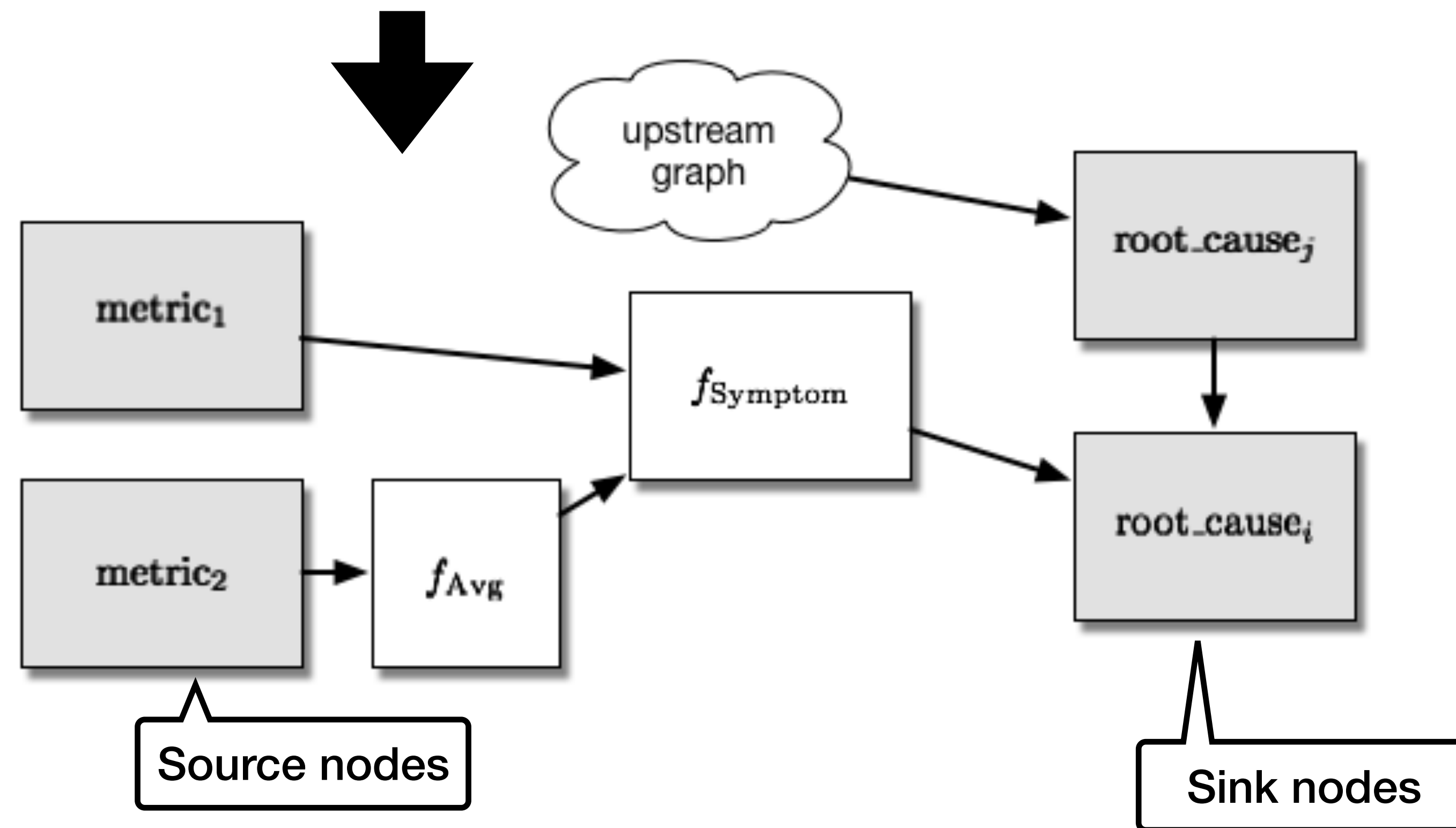
- Observability emits hundreds of fine-grained metrics
- RCA distills down to current problem(s)
- What is a root cause?

$$\text{root_cause}_i = f_{rca}(\text{symptom}_k, \text{root_cause}_j)$$

$$\text{symptom}_k = f_{\text{symptom}}(\text{metric}_1, f_{\text{avg}}(\text{metric}_2))$$

RCA as data flow graph

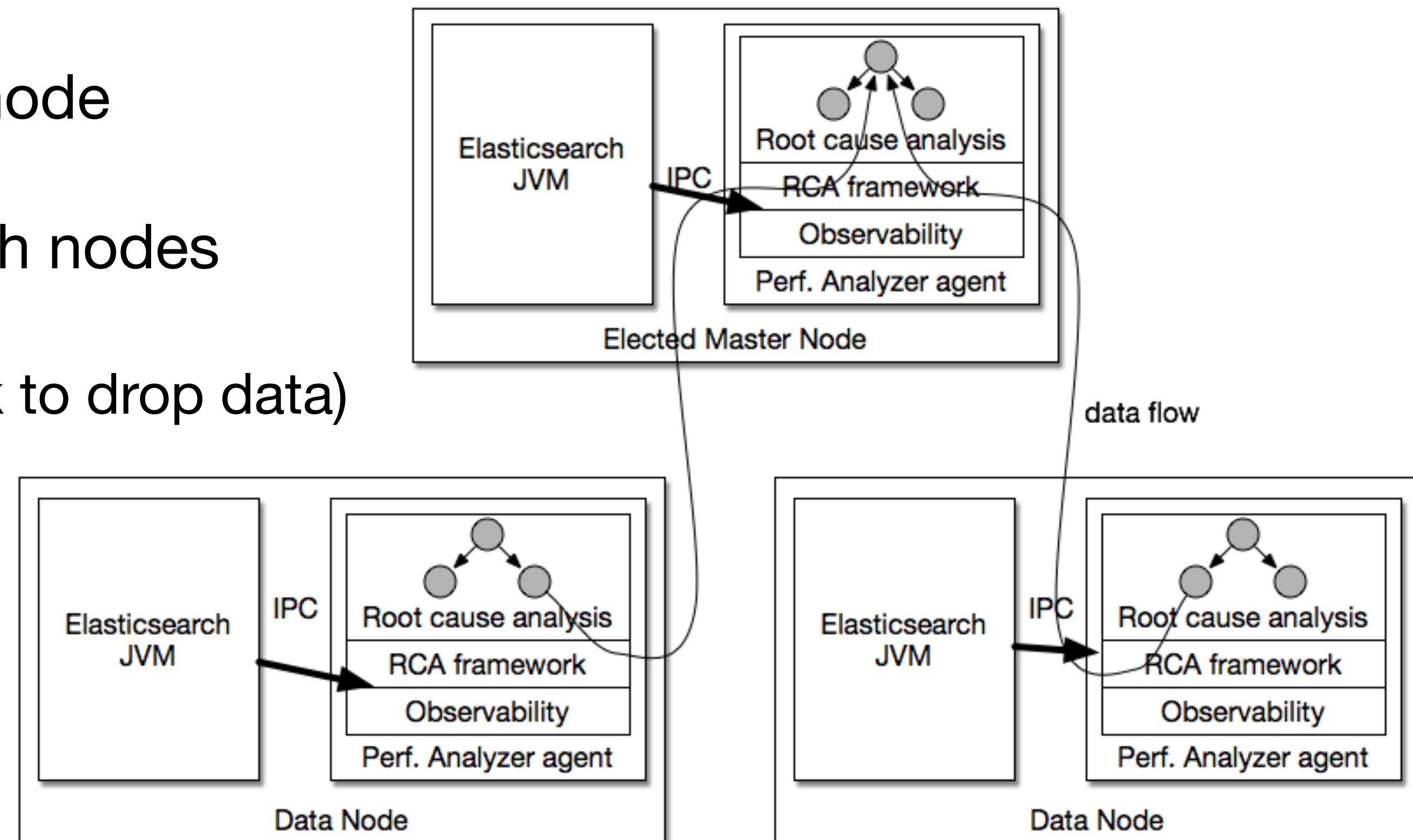
$$\text{root_cause}_i = f_{rca}(\text{symptom}_k, \text{root_cause}_j)$$
$$\text{symptom}_k = f_{\text{symptom}}(\text{metric}_1, f_{\text{avg}}(\text{metric}_2))$$



- Root cause can span multiple cluster nodes
- RCA node is 1-1 with cluster node
- Graph of all RCAs is a DAG

RCA framework

- Implements real time streaming runtime for the RCA graph
- *schedules* and *executes* each graph node
- *passes* opaque objects between graph nodes
 - ordered and at-most-once delivery (ok to drop data)
 - staleness and loss: via timestamps
- edge setup: nodes subscribe
- framework de-dups compute



Graph node definition

- Implement method to execute:

```
class HighShardCpuRca extends RCA {  
    HighShardCpuRca (evaluationIntervalSeconds);  
    @Override  
    FlowUnit evaluate (Map<Class, FlowUnit> dependencies);  
}
```

- Specify graph edges

```
class RCAGraph extends AnalysisGraph {  
    @Override  
    void construct ();  
}
```

- Threshold and parameter inputs configurable externally: per-node, per-type

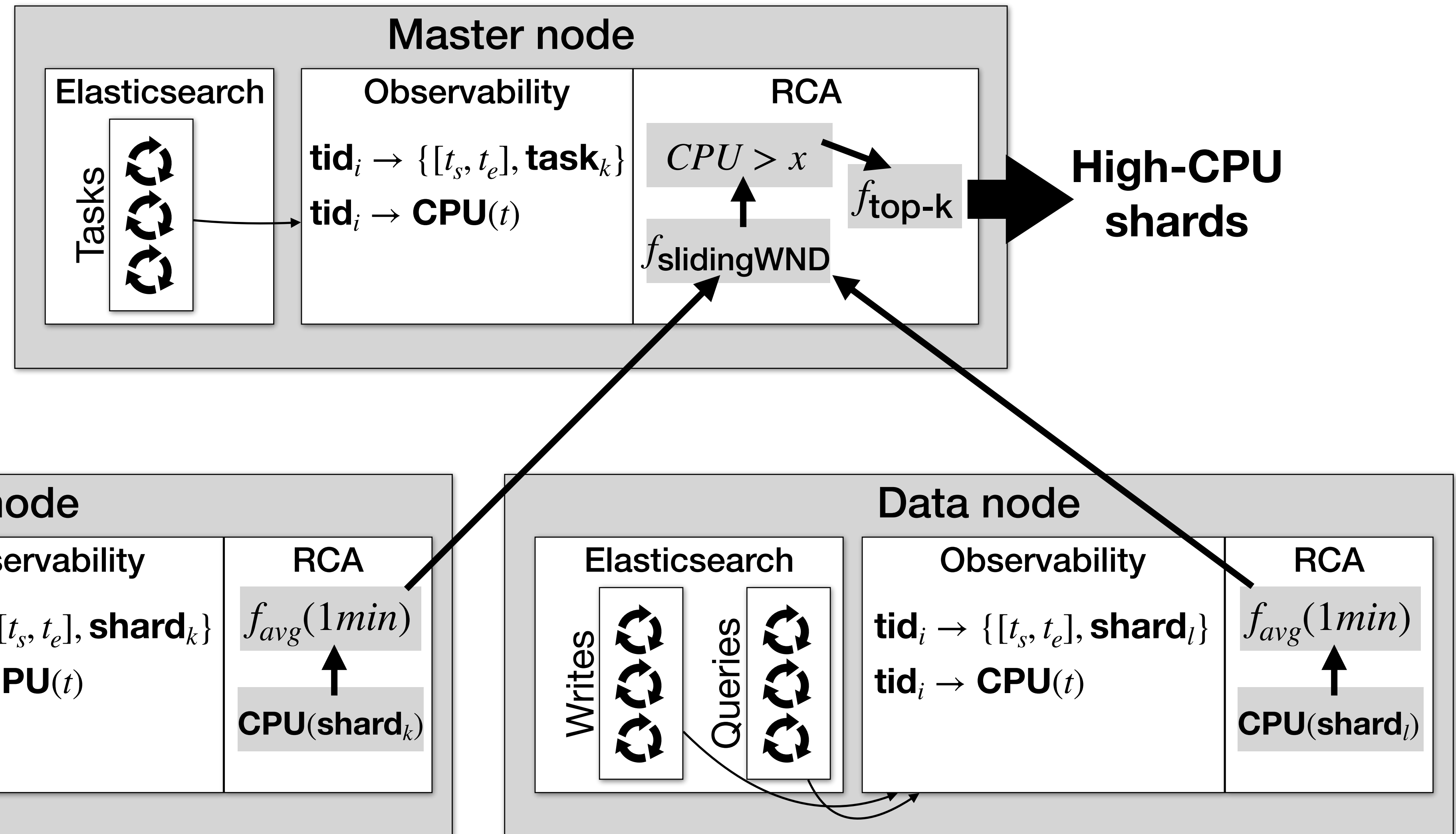
RCA API

- Output of RCA: returns last 5s of diagnoses

GetRCAs ()

RCA name	Time since first diagnosis	Context (e.g., metrics, thresholds ...)

Example RCA: High-CPU Shards



RFC for Root Cause Analysis

- Posted on Open Distro Github
<https://github.com/opendistro-for-elasticsearch/performance-analyzer/blob/master/rca/>
- Contributions and comments welcome!
<https://github.com/opendistro-for-elasticsearch/performance-analyzer/issues/73>

RFC: Root Cause Analysis

SEPTEMBER 2019

Introduction

The Open Distro for Elasticsearch Performance Analyzer captures Elasticsearch and JVM layer activity, and lower-level resource usage (e.g., disk, network, CPU and memory) of these activities. Based on this instrumentation, Performance Analyzer computes and exposes *diagnostic* metrics, with the goal of enabling Elasticsearch users and administrators to measure and understand bottlenecks in their Elasticsearch clusters. The Open Distro for Elasticsearch PerfTop client provides real time visualization of these diagnostic metrics to surface bottlenecks to Elasticsearch users and operators.

This RFC proposes a design for a framework that builds on the Performance Analyzer to support root cause analysis (RCA) of performance and reliability problems for Elasticsearch instances. This framework would conduct real time root cause analysis of such problems using Performance Analyzer metrics. Root cause analysis can significantly improve operations, administration and provisioning of Elasticsearch clusters, and it can enable Elasticsearch client teams to tune their workloads to reduce errors.

Root Cause Analysis

The first step is to define what we mean by a "root cause". We define a root cause as a function of one or more *symptoms*. A symptom is an operation over metrics or *other* symptoms. Root causes may also be a function of other root causes. Note that these operations may involve aggregations; for example, a symptom could consume a time average of a metric. In addition, for confidence, a root cause could be a computation over a sufficiently long window of time. This definition does not allow for cycles in the dependency graph between metrics and root causes. The following equations show an example of these relationships:

$$\begin{aligned} \text{root_cause}_i &= f_{\text{RCA}}(\text{symptom}_k, \text{root_cause}_j) \\ \text{symptom}_k &= f_{\text{Symptom}}(\text{metric}_1, f_{\text{Avg}}(\text{metric}_2)) \end{aligned}$$

Real Time Performance Diagnosis in Distributed Databases

Partha Kanuparth, Alolita Sharma (AWS)
ApacheCon 2019

